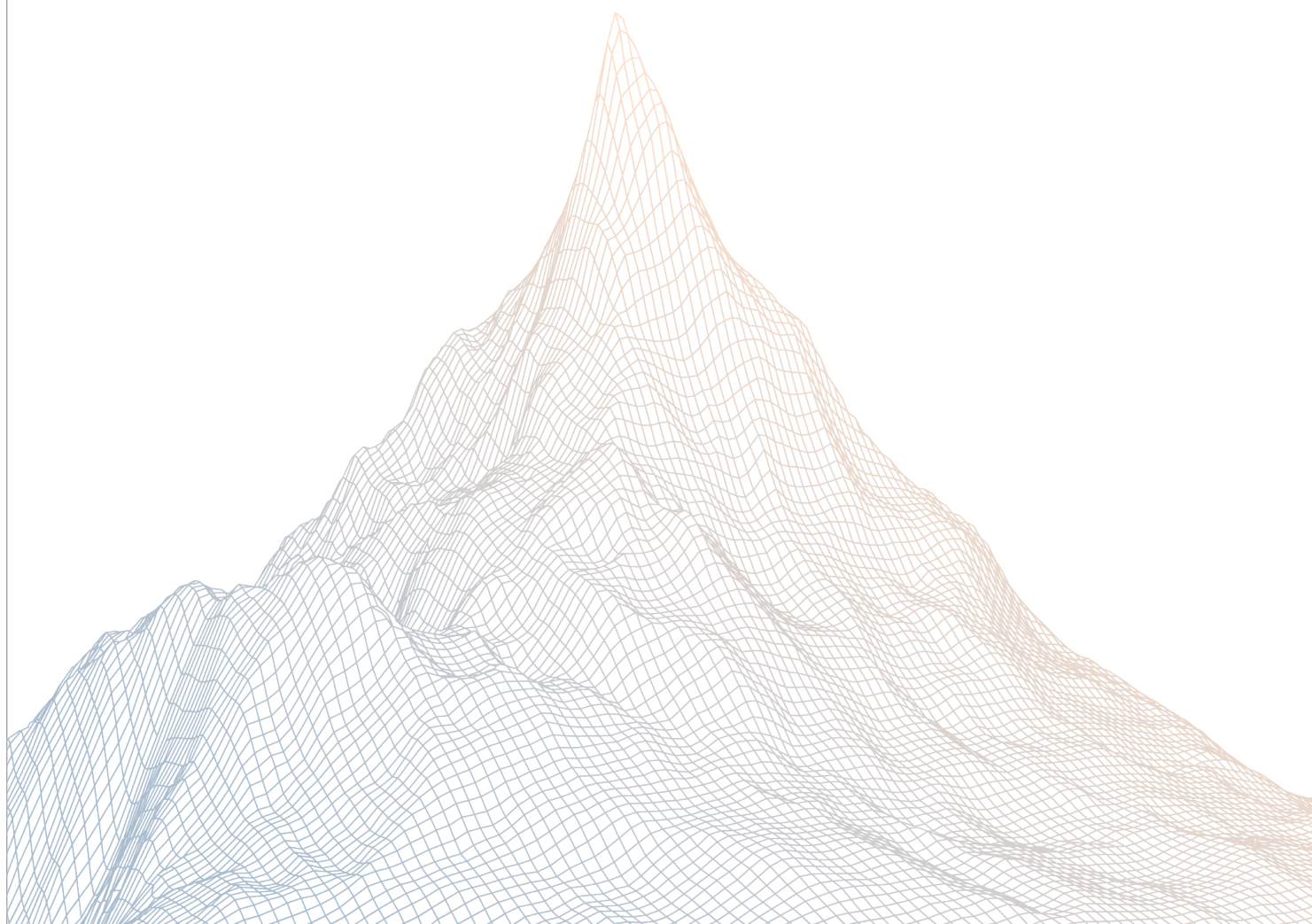


Bridge

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

November 26th to November 28th, 2025

AUDITED BY:

cccZ
said

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Bridge	4
2.2	Scope	4
2.3	Audit Timeline	6
2.4	Issues Found	6
3	Findings Summary	6
4	Findings	7
4.1	Medium Risk	8
4.2	Low Risk	17

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Bridge

An entirely new payments platform, built with stablecoins, to simplify global money movement.

2.2 Scope

The engagement involved a review of the following targets:

Target	erc20-stablecoin-template
---------------	---------------------------

Repository	https://github.com/withbridge/erc20-stablecoin-template
-------------------	---

Commit Hash	2a6ec1c10b036350b6114754c3212567d53d9cc8
--------------------	--

Files	Changes in PR #3
--------------	------------------

Target	auth-registry
---------------	---------------

Repository	https://github.com/withbridge/auth-registry
-------------------	---

Commit Hash	19a41db72625cfe494be2dfd5676c95f348dd13e
--------------------	--

Files	Changes in PR #7
--------------	------------------

Target	erc20-stablecoin-template
Repository	https://github.com/withbridge/erc20-stablecoin-template/pull/5
Commit Hash	01c2e6d9f1bb4cbaa93d7c8e5d81bba7b896cec8
Files	Changes in PR #5

2.3 Audit Timeline

November 26, 2025	Audit start
November 28, 2025	Audit end
December 11, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	4
Low Risk	2
Informational	0
Total Issues	6

3

Findings Summary

ID	Description	Status
M-1	completeMigrationToWrapped can be griefed by dust transfers	Acknowledged
M-2	TokenAuthority.burn() and unwrap() should transfer tokens first	Resolved
M-3	The total supply of StablecoinTemplateV3 may be greater than _maxSupply	Acknowledged
M-4	_update() does not check if the recipient is on the blacklist	Acknowledged
L-1	reinitialize zeroes _maxSupply, breaking minting on ReserveLedger	Resolved
L-2	TokenAuthority.setMinterAllowance() race condition	Acknowledged

4

Findings

4.1 Medium Risk

A total of 4 medium risk findings were identified.

[M-1] `completeMigrationToWrapped` can be grieved by dust transfers

SEVERITY: Medium

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: High

Target

- [StablecoinTemplateV3.sol#L122](#)

Description:

`completeMigrationToWrapped` sets `_migrationToWrappedCompleted = true` and then requires `RESERVE_LEDGER_ADDRESS.balanceOf(address(this)) = totalSupply()`. Any `RESERVE_LEDGER_ADDRESS` holder can grief this by transferring a dust of reserve tokens directly to the stablecoin contract, making the balance exceed `totalSupply()` and preventing migration.

```
function completeMigrationToWrapped() public onlyRole(DEFAULT_ADMIN_ROLE) {
    StablecoinTemplateV3StorageLib.getStorage()._migrationToWrappedCompleted
    = true;
    require(
>>>    RESERVE_LEDGER_ADDRESS.balanceOf(address(this)) = totalSupply(),
        ReserveLedgerBalanceMismatch()
    );
    emit MigrationHasCompleted(msg.sender);
}
```

Recommendations:

Consider updating the following check:

```
require(
```



```
RESERVE_LEDGER_ADDRESS.balanceOf(address(this)) = totalSupply(),  
RESERVE_LEDGER_ADDRESS.balanceOf(address(this)) ≥ totalSupply(),  
ReserveLedgerBalanceMismatch()  
);
```

Bridge: Acknowledged. Given we are the only ones who can mint, in the event that we have too much balance while doing the migration we can always mint more tokens before completing the migration. we want to ensure we are properly collateralized instead of over.

[M-2] `TokenAuthority.burn()` and `unwrap()` should transfer tokens first

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [TokenAuthority.sol#L134-L167](#)

Description:

`TokenAuthority.burn()` and `unwrap()` require the caller to first transfer tokens to `TokenAuthority` before calling `TokenAuthority.burn()` and `unwrap()`.

Consider the `BURNER_ROLE` transferring some tokens to `TokenAuthority` and planning to burn them, while the `UNWRAPPER_ROLE` can unwrap them after the tokens have been transferred to `TokenAuthority`.

```
function burn(address stablecoinContract, uint256 amount)
    public onlyRole(BURNER_ROLE) {
    if (stablecoinContract == RESERVE_LEDGER_TOKEN) {
        IERC20BurnMint(RESERVE_LEDGER_TOKEN).burn(amount);
    } else {
        IERC20WrapUnwrap(stablecoinContract).unwrap(amount);
        IERC20BurnMint(RESERVE_LEDGER_TOKEN).burn(amount);
    }

    emit Burn(msg.sender, stablecoinContract, amount);
}
...
function unwrap(address stablecoinContract, uint256 amount)
    public onlyRole(UNWRAPPER_ROLE) {
    require(stablecoinContract != RESERVE_LEDGER_TOKEN,
        CannotUnwrapReserveLedgerToken());

    // Unwrap the wrapped stablecoin, which will send underlying
    RESERVE_LEDGER_TOKEN to this
    // contract
    IERC20WrapUnwrap(stablecoinContract).unwrap(amount);
}
```

```
// Transfer the received RESERVE_LEDGER_TOKEN to the sender
IERC20BurnMint(RESERVE_LEDGER_TOKEN).transfer(msg.sender, amount);

emit Unwrap(msg.sender, stablecoinContract, amount);
}
```

Recommendations:

It is recommended to transfer the caller's tokens to TokenAuthority in `TokenAuthority.burn()` and `unwrap()`.

```
function burn(address stablecoinContract, uint256 amount)
public onlyRole(BURNER_ROLE) {
    IERC20BurnMint(stablecoinContract).transferFrom(msg.sender, address(
        this), amount);
    if (stablecoinContract == RESERVE_LEDGER_TOKEN) {
        IERC20BurnMint(RESERVE_LEDGER_TOKEN).burn(amount);
    } else {
        IERC20WrapUnwrap(stablecoinContract).unwrap(amount);
        IERC20BurnMint(RESERVE_LEDGER_TOKEN).burn(amount);
    }

    emit Burn(msg.sender, stablecoinContract, amount);
}

...

function unwrap(address stablecoinContract, uint256 amount)
public onlyRole(UNWRAPPER_ROLE) {
    require(stablecoinContract != RESERVE_LEDGER_TOKEN,
        CannotUnwrapReserveLedgerToken());
    IERC20BurnMint(stablecoinContract).transferFrom(msg.sender, address(
        this), amount);

    // Unwrap the wrapped stablecoin, which will send underlying
    // RESERVE_LEDGER_TOKEN to this
    // contract
    IERC20WrapUnwrap(stablecoinContract).unwrap(amount);

    // Transfer the received RESERVE_LEDGER_TOKEN to the sender
    IERC20BurnMint(RESERVE_LEDGER_TOKEN).transfer(msg.sender, amount);

    emit Unwrap(msg.sender, stablecoinContract, amount);
}
```

Bridge: Resolved with [PR-6](#).

Zenith: Verified.

[M-3] The total supply of StablecoinTemplateV3 may be greater than `_maxSupply`

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [StablecoinTemplateV3.sol#L40-L49](#)

Description:

`StablecoinTemplateV3.wrap()` does not require the total supply to be less than `_maxSupply`, which causes the total supply of `StablecoinTemplateV3` may be greater than `_maxSupply`.

```
function wrap(address to, uint256 amount) public {
    require(amount > 0, AmountCannotBeZero());
    require(isMintRecipient(to), AccountNotValidRecipient());

    RESERVE_LEDGER_ADDRESS.safeTransferFrom(msg.sender, address(this),
    amount);

    _mint(to, amount);

    emit Wrapped(amount, to, msg.sender);
}
```

Recommendations:

```
function wrap(address to, uint256 amount) public {
    require(amount > 0, AmountCannotBeZero());
    require(isMintRecipient(to), AccountNotValidRecipient());
    require(totalSupply() + amount ≤ $_maxSupply, MaxSupplyExceeded());

    RESERVE_LEDGER_ADDRESS.safeTransferFrom(msg.sender, address(this),
    amount);

    _mint(to, amount);
}
```

```
emit Wrapped(amount, to, msg.sender);  
}
```

Bridge: Acknowledged. This is fine since the reason for this is to disallow fat finger inputs for large mints. Because it requires users to have the equivalent amount of reserve tokens we can push this down to the reserve ledger.

[M-4] `_update()` does not check if the recipient is on the blacklist

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [StablecoinTemplateV3Base.sol#L296-L303](#)

Description:

After the change, `_update()` forgets to call `whenNotInBlockedList(to)`, causing token transfers to no longer check if the recipient is on the blacklist.

```
function _update(address from, address to, uint256 amount)
    internal
    override
    whenNotPaused
    whenNotInBlockedList(from)
{
    super._update(from, to, amount);
}
```

Recommendations:

```
function _update(address from, address to, uint256 amount)
    internal
    override
    whenNotPaused
    whenNotInBlockedList(from)
    whenNotInBlockedList(to)
{
    super._update(from, to, amount);
}
```

Bridge: Acknowledged. This was an explicit decision, following what's done for USDT we want to allow for token to be transferred to a blocked account and then they won't be able

to move out. This allows us to immediately burn from their account vs potentially having the block frontrun by a transfer.

4.2 Low Risk

A total of 2 low risk findings were identified.

[L-1] reinitialize zeroes _maxSupply, breaking minting on ReserveLedger

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StablecoinTemplateV3Base.sol#L115](#)

Description:

Every call to `_initialize` sets `StablecoinTemplateV3Storage._maxSupply` to 0, and `reinitialize` invokes it. This is not an issue for `StablecoinTemplateV3`, but for `ReserveLedger`, reinitializing it will brick minting until `_maxSupply` is reconfigured by the admin.

```
function reinitialize(
    string calldata _name,
    string calldata _symbol,
    uint8 __decimals,
    address admin,
    uint64 _transferPolicyId,
    uint64 _mintRecipientPolicyId
) public reinitializer(2) onlyOwnerOrAdmin {
    _initialize(_name, _symbol, __decimals, admin, _transferPolicyId,
        _mintRecipientPolicyId);
}

function _initialize(
    string calldata _name,
    string calldata _symbol,
    uint8 __decimals,
    address admin,
    uint64 _transferPolicyId,
    uint64 _mintRecipientPolicyId
```

```
) internal onlyInitializing {  
    require(admin != address(0), AdminCannotBeZeroAddress());  
  
    StablecoinTemplateV3Storage storage $  
    = StablecoinTemplateV3StorageLib.getStorage();  
  
    $_maxSupply = 0;  
    $_decimals = __decimals;  
  
    $_transferPolicyId = _transferPolicyId;  
    $_mintRecipientPolicyId = _mintRecipientPolicyId;  
  
    __ERC20_init(_name, _symbol);  
    __Pausable_init();  
    __Ownable_init(admin);  
    __AccessControl_init();  
    __ERC20Permit_init(_name);  
    __UUPSUpgradeable_init();  
  
    _grantRole(DEFAULT_ADMIN_ROLE, admin);  
}
```

Recommendations:

Allow reinitialize caller to preserve the previous `_maxSupply`.

Bridge: Resolved with [PR-7](#).

Zenith: Verified.

[L-2] `TokenAuthority.setMinterAllowance()` race condition

SEVERITY: Low

IMPACT: High

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [TokenAuthority.sol#L214-L222](#)

Description:

When the `MINT_RATE_LIMIT_SETTER_ROLE` calls `setMinterAllowance()` to update the minter's `minterAllowances`, it may be suffering from frontrun attack.

1. Consider `minterAllowances[alice] = 5000`, and the `MINT_RATE_LIMIT_SETTER_ROLE` plans to set it to 10000.
2. Alice front runs the `MINT_RATE_LIMIT_SETTER_ROLE`'s call to `setMinterAllowance(alice, 10000)`, minting 5000 tokens.
3. Later, when Alice's `minterAllowances` is updated to 10000, Alice actually has 15000 `minterAllowances`.

```
function setMinterAllowance(address stablecoinContract, address minter,
    uint256 minterAllowance)
    public
    onlyRole(MINT_RATE_LIMIT_SETTER_ROLE)
{
    require(minterAllowance < type(uint256).max / 2,
        AmountExceedsAbsoluteMax());
    minterAllowances[stablecoinContract][minter] = minterAllowance;

    emit MinterAllowanceSet(msg.sender, stablecoinContract, minter,
        minterAllowance);
}
```

Recommendations:

It is recommended to use `increaseMinterAllowance()` and `decreaseMinterAllowance` to manage `minterAllowances`, making `minterAllowances` increase or decrease from their

original values.

Bridge: Acknowledged. It's easier for us to reason about the set allowance (similar to how `erc20` approvals function) and adding both functions would increase the size of the contract. We're less concerned with the race condition as all minters are our own wallets and this is a mechanism intended to be used to stop our code from minting too much accidentally.